

Praxis: Boot und Initialisierung untersuchen

Bootphasen auf einem x86-System messen, belegen und erklären

5.03

Die Bootanalyse als sichere Laborübung vorbereiten

LABORUMGEBUNG

VM oder Testsystem

Aktuellen Boot zunächst nur lesend untersuchen. Keine Produktionsmaschine verwenden.

WERKZEUGE

Logs und Browser

systemd, Journal und ein Browser für die SVG-Ausgabe müssen verfügbar sein.

BERECHTIGUNGEN

Ohne Neustart starten

Pflichtteil als normaler Benutzer. sudo nur für eingeschränkte Kernel-Logs.

Bootparameter erst in der optionalen Vertiefung und nur einmalig im Bootmenü ändern.

Werkzeuge und Ausgangszustand prüfen

BASH · BESTANDSAUFNAHME

```
$ uname -r  
$ command -v systemd-analyze  
$ command -v journalctl  
$ test -d /sys/firmware/efi \  
    && echo UEFI || echo BIOS/CSM  
$ cat /proc/cmdline
```

01 Systembezug

Kernelrelease und vorhandene Analysewerkzeuge festhalten.

02 Bootmodus

UEFI und BIOS/CSM liefern unterschiedliche Firmware-Telemetrie.

Kernelversion, Bootmodus und Kommandozeile gehören in jeden Messbericht.

Bootmodus und Kernelparameter dokumentieren

/PROC/CMDLINE · BOOTVERTRAG

```
$ cat /proc/cmdline
$ findmnt /
$ lsblk -f
$ ps -p 1 -o pid,comm,args

# Werte und Zuordnung notieren
```

Root-Pfad

root= mit Blockgerät, UUID, LVM oder Device Mapper verbinden.

PID 1

Init-System und tatsächlichen Prozess nach dem Kernelübergang belegen.

root=, init=, quiet und resume= verändern unterschiedliche Phasen des Bootpfads.

Bootphasen mit systemd-analyze messen

SYSTEMD · GESAMTZEIT

```
$ systemd-analyze time

# Ergebnisse notieren:
# firmware · loader · kernel
# initrd · userspace

$ systemd-analyze --no-pager
```

01 Messgrenze

Das gemeldete Target ist nicht automatisch der fachliche Betriebszustand.

02 Fehlende Werte

Nicht jede Plattform liefert Firmware- und Loader-Zeitstempel.

Fehlende Firmwarezeit bedeutet fehlende Telemetrie, nicht null Sekunden.

Messwerte nach ihrer Bootphase einordnen

Firmware

UEFI bis zur Übergabe an den Bootloader

Bootloader

Bootauswahl sowie Laden von Kernel und initramfs

Kernel

Architekturstart, Initcalls und Root-Vorbereitung

User Space

PID 1, Units und fachlicher Zielzustand

Nur Messungen mit demselben Start- und Endpunkt sind vergleichbar.

Kernelmeldungen als monotone Zeitlinie lesen

KERNELLOG · AKTUELLER BOOT

```
$ journalctl -b -k \
    -o short-monotonic --no-pager

# Alternative:
$ sudo dmesg --time-format=delta

# Suche: ACPI · PCI · NVMe/SATA · rootfs
```

01 Zeitlücken

Abstände zwischen
Meldungen als Indiz
markieren.

02 Subsysteme

Meldungen Architektur, Bus,
Treiber und Root-Pfad
zuordnen.

Die Meldung nach einer Lücke ist ein Hinweis, aber kein Kausalitätsbeweis.

Die zeitkritische User-Space-Kette bestimmen

SYSTEMD · ABHÄNGIGKEITEN

```
$ systemd-analyze critical-chain  
$ systemd-analyze blame --no-pager | head -20  
$ systemctl status NAME.service  
$ journalctl -b -u NAME.service --no-pager
```

01 Kritischer Pfad

Abhängigkeiten bis zum Zielzustand verfolgen.

02 Warteursache

Journal und Unit-Beziehungen gemeinsam prüfen.

Eine lange Unit verkürzt den Boot nur dann, wenn sie auf der kritischen Kette liegt.

Vier typische Messfehler vermeiden

Einzellauf

Streuung bleibt unsichtbar

blame allein

Parallelität und Warteabhängigkeiten fehlen

Mehr Konsolenausgabe

Logging verändert das gemessene Timing

Target = betriebsbereit

Der fachliche Endpunkt kann später liegen

Mindestens zehn vergleichbare Läufe durchführen und Median sowie Streuung dokumentieren.

Die parallele Aktivierung als SVG auswerten

SYSTEMD · VISUELLE ZEITLEISTE

```
$ systemd-analyze plot > boot.svg
$ systemd-analyze critical-chain \
    > critical-chain.txt

# boot.svg im Browser öffnen
# Start, Ende und Überlappung markieren
```

01 Parallelität lesen

Überlappende Units von der kritischen Kette unterscheiden.

02 Evidenz sichern

SVG, Kette und zugehörige Journalstellen gemeinsam ablegen.

Plot, critical-chain und Journal ergänzen sich. Kein einzelner Befund genügt.

Optional: Langsame Kernel-Initcalls messen

EINMALIGER GRUB-PARAMETER

```
initcall_debug log_buf_len=16M

# Nach dem Neustart:
$ sudo dmesg > boot-initcalls.log
$ grep 'initcall .* returned' \
    boot-initcalls.log
```

01 Testsystem verwenden

Parameter nur für einen Start im GRUB-Menü ergänzen.

02 Messaufwand beachten

Zusätzliche Meldungen beeinflussen den beobachteten Boot.

Nach der Analyse erneut ohne Debugparameter messen.

Auffällige Zeiten systematisch eingrenzen

DIAGNOSE · BEFUND MIT GEGENPROBE

```
$ perl scripts/bootgraph.pl \  
    < boot-initcalls.log > initcalls.svg  
$ systemctl show NAME.service \  
    -p After -p Requires -p Wants
```

KERNELTREIBER

Initcall + Quelle

Funktion einem Treiber oder Subsystem zuordnen.

HARDWARE-WARTEZEIT

ACPI · PCI · Storage

Loglücke mit Geräteerkennung und Firmwaremeldung korrelieren.

USER SPACE

Unit + Abhängigkeit

Kritische Kette und Journal gemeinsam prüfen.

Befund dokumentieren und nächste Messung planen

01 Messpunkt festlegen

Kalt- oder Warmstart, Startsignal und fachlichen Endpunkt dokumentieren.

02 Beobachtung sichern

Phasenwerte, Logs, SVG und auffällige Zeitmarken ablegen.

03 Interpretation begrenzen

Gemessene Fakten klar von Ursachenhypothesen trennen.

04 Nächsten Test definieren

Genau eine Variable ändern und unter gleichen Bedingungen erneut messen.

Ein belastbarer Befund nennt Messgrenze, Evidenz, Unsicherheit und nächsten Test.